

IE3142 DevOps Security — Building and Securing a DevSecOps Pipeline

Executive Summary

This project applies a practical DevSecOps lifecycle to OWASP NodeGoat, an intentionally vulnerable Node.js/Express and MongoDB application designed for security education. The application was selected because it satisfies the assignment requirement for multiple communicating components, can run locally with Docker Compose, and contains realistic vulnerability classes that can be demonstrated and remediated without attacking an external system. The project treats security as a continuous engineering activity rather than a final checklist: vulnerabilities are identified in source code, mapped to threats, fixed with secure coding controls, and then placed behind automated CI/CD security gates.

The final architecture contains a NodeGoat web service and a MongoDB service on a private Docker Compose network. Only the web service is published to localhost. The repository also contains GitHub Actions automation for SAST, dependency scanning, secret detection and container image scanning. Secrets are supplied through environment variables locally and GitHub encrypted secrets in CI. The final runtime code removes several intentionally vulnerable patterns while retaining the original educational application structure.

1. System Overview and Architecture

NodeGoat uses Node.js with Express, MongoDB as its data store, Swig templates, and Express sessions. The browser communicates with the web service over HTTP on local port 4000. The web service communicates with MongoDB over the Compose network; MongoDB is not published to the host.

The architecture creates two important runtime trust boundaries. The first is between the untrusted browser and Express: all form fields, query parameters and cookies must be treated as attacker-controlled. The second is between Express and MongoDB: values originating from a request must not be converted into executable query syntax. The CI pipeline forms a separate build trust boundary because source code and pull requests are inputs to automated security checks.

Containerisation uses the existing application Dockerfile and a root-level `docker-compose.yml`. The web container runs as a non-root Node user, uses a read-only filesystem with a temporary filesystem for `/tmp`, and has `no-new-privileges` enabled. MongoDB is reachable only internally. These measures reduce the impact of container compromise but do not replace application-level security.

2. STRIDE Threat Modelling and Risk Assessment

A 3×3 likelihood/impact model was used. Likelihood and impact are each rated from 1 to 3, and the product gives the risk score. Eight application-specific threats were identified.

The first threat is server-side JavaScript injection in the contribution endpoint. The original handler used `eval()` on three values supplied directly by the request. This is a Tampering threat with high impact because successful input can change server-side execution. The control is strict decimal parsing in

`contributions.js`, reinforced by a Semgrep rule.

The second threat is MongoDB JavaScript injection. The original allocation query built a `$where` expression containing the request's `threshold`. This is a Tampering/Elevation threat because a user can influence database logic. The remediation replaces `$where` with a typed `$gt` comparison.

The third threat is insecure direct object reference. The allocation route trusted a `userId` path parameter rather than the authenticated session. This creates an Information Disclosure/Elevation threat: one user can request another user's allocation data. The route now obtains the identity exclusively from the session.

The fourth threat is SSRF in the research endpoint. The original application accepted a client-controlled destination URL and used the server as the HTTP client. The fixed implementation ignores the destination supplied by the browser, uses a fixed application-approved destination, and validates the stock symbol.

Additional threats include ReDoS in bank-routing validation, plaintext password storage, session fixation, and missing CSRF protection. These findings show why STRIDE should be applied to concrete data flows rather than copied as generic categories.

3. Secure Coding: Exploit-and-Fix

Four primary exploit-and-fix cases were selected.

VUL-01: Server-side JavaScript injection

The baseline `contributions.js` executed `req.body.preTax`, `afterTax`, and `roth` using `eval()`. A local authenticated test can place an expression such as `process.version` into a contribution field. A visible expression result demonstrates that request data reached the JavaScript interpreter. No operating-system command is required to prove the issue.

The remediation replaces dynamic evaluation with a bounded numeric parser. Only one- or two-digit decimal values are accepted, converted with `Number.parseInt`, and checked for a valid contribution total. Repeating the original request against the fixed application must result in a validation error rather than expression evaluation.

VUL-02: MongoDB ``$where`` injection

The baseline allocation DAO constructed a JavaScript `$where` expression using the threshold parameter. A crafted value such as `0' || '1' == '1` can change the logical predicate and broaden the result set. This demonstrates that user data has become executable database logic.

The fix validates the threshold as an integer between 0 and 99 and constructs a normal MongoDB query using `{ stocks: { $gt: parsedThreshold } }`. The same crafted input is now rejected as invalid.

VUL-03: IDOR/BOLA

The original allocation route used `/allocations/:userId` as the data identity. A logged-in test user could change the path to another seeded user's ID. If another user's allocation was displayed, authorization was

proven to depend on a client-controlled identifier.

The fix removes trust in the path identity and uses `req.session.userId`. Repeating the modified URL therefore cannot change which user's allocation is selected.

VUL-04: SSRF

The research route originally concatenated a browser-supplied URL and stock symbol and passed the resulting address to `needle.get()`. In the authorized local lab, the before-state can be demonstrated using a controlled local HTTP service and observing a request arriving at that service.

The fix removes the client-controlled destination and restricts the stock symbol to a small allow-list pattern. The exact previous URL parameter is therefore ignored. This demonstrates the principle that SSRF prevention should constrain the destination rather than merely validate the input superficially.

Before/after screenshots and scan counts are intentionally not fabricated in this package. The evidence directory contains the required structure and instructions; the group must execute the baseline and fixed builds and capture genuine results.

4. CI/CD Pipeline and Security Automation

The GitHub Actions pipeline is triggered on pushes, pull requests and manual execution. It first installs the Node.js dependencies and executes a security regression test that checks the fixed source for the four critical patterns. Semgrep then performs SAST using a project-specific ruleset. The SAST gate is configured to return a failing exit code when a high-confidence rule matches.

Dependency scanning uses `npm audit`. The final workflow uses a critical threshold so that known lower-severity legacy dependency findings can be recorded without making the educational project permanently unbuildable. This threshold should be reviewed by the group after the actual audit result is available.

Gitleaks scans the repository for accidentally committed credentials. Trivy scans the built Docker image for high and critical vulnerabilities and is configured to fail the job when such findings are present. The workflow therefore contains both source-level and artifact-level controls.

A separate manually triggered workflow demonstrates a genuinely failing gate without committing a permanent vulnerable application file. It creates a temporary JavaScript fixture containing `eval(req.body.input)` and runs the same Semgrep configuration with `--error`. The expected result is a failed workflow. The group should run this workflow once, capture the failed job, then run the normal workflow on the fixed repository and capture the successful run.

OWASP ZAP is included as an optional manual DAST workflow. It is not counted as one of the four mandatory gates.

5. Secrets Management

The original configuration contained a hardcoded session secret and the seed script contained plaintext demo passwords. Both practices were removed from current runtime configuration.

The application now requires `SESSION_SECRET` and seed passwords to be supplied through the environment. The Compose file uses required variable expansion, while `.env.example` contains placeholders only. The seed script hashes passwords with Node.js `crypto.scryptSync` using a random salt and stores only the salt and derived hash.

For CI, the same values should be created as GitHub Actions encrypted repository secrets. They are referenced through the workflow rather than written to source. Gitleaks provides an independent check against accidental secret commits. The `.gitignore` excludes real `.env` files.

6. Industry Trend and Case Study Connection

Modern DevSecOps practice increasingly treats security checks as automated quality gates close to the point where code changes are introduced. The project follows this principle by combining SAST, software composition analysis, secret detection and container scanning in one pipeline. A useful lesson from real-world software supply-chain incidents is that application security is not limited to application source code: dependencies, build artifacts, credentials and CI configuration can all become attack paths.

The project also demonstrates a second industry trend: policy should be expressed as executable controls. Instead of documenting “do not use dangerous evaluation,” the Semgrep rule makes the requirement enforceable. Instead of stating that secrets should not be committed, Gitleaks automatically checks the repository. This reduces reliance on individual memory and makes the security process repeatable.

7. Reflection and Future Improvements

The strongest improvement opportunity is evidence automation. The current package defines the exploit/fix methodology and pipeline, but the group must still capture real runtime screenshots and scan results. With more time, security regression tests could automatically exercise the vulnerable endpoints and verify that authorization, injection and validation controls remain effective.

Dependency modernization is another priority. NodeGoat is an intentionally old educational application, so its dependency graph requires careful compatibility testing before upgrades. A future version could replace obsolete packages, move to a supported MongoDB driver, use a maintained CSRF library, and introduce a modern template framework.

The pipeline could also add SBOM generation, dependency review for pull requests, IaC scanning and signed container provenance. Runtime logging and centralized alerting could be improved while ensuring that sensitive values are never logged.

8. Individual Contribution and AI Usage

The group must complete the contribution statement with the actual four members, student IDs, roles and signatures. AI usage must be disclosed honestly. For this package, AI assistance was used for project planning, code-review reasoning, security-control design, documentation structure and draft implementation support. The group must review, test and understand every generated change and must not claim that

AI-generated evidence is genuine execution evidence.

Conclusion

The project implements the core DevSecOps lifecycle required by IE3142: a real open-source application is containerised, threats are modelled against its architecture, concrete vulnerabilities are mapped to secure coding controls, secrets are externalised, and automated security gates are integrated into CI/CD. The remaining submission-critical work is execution evidence: the group must run the baseline exploits, apply the fixed build, execute the security scanners, capture the blocking and passing pipeline runs, complete the ethical clearance and contribution information, and export the final report with authentic screenshots and tool results.

References

- [1] OWASP, "NodeGoat," OWASP, Apache License 2.0 project documentation.
- [2] OWASP, "OWASP Top 10:2021," Open Worldwide Application Security Project.
- [3] OWASP, "OWASP Application Security Verification Standard," Open Worldwide Application Security Project.
- [4] Microsoft, "Threat Modeling Tool — STRIDE threat categories," Microsoft Security documentation.
- [5] Semgrep, "Semgrep Documentation — Static Analysis and CI," Semgrep documentation.
- [6] Aqua Security, "Trivy Documentation," container and artifact vulnerability scanner documentation.
- [7] Gitleaks, "Gitleaks Documentation," secret detection documentation.
- [8] OWASP, "OWASP ZAP," open-source web application security scanner documentation.
- [9] NIST, "Secure Software Development Framework (SSDF), SP 800-218," National Institute of Standards and Technology.